UNIVERSIDADE PAULISTA – UNIP
Av. Alberto Benassi, 200 - Parque das Laranjeiras - Araraquara - SP
CEP 14804-300 - Tel.: (16) 3336-1800

ATIVIDADES PRÁTICAS SUPERVISIONADAS

ENGENHARIA DE ESCALABILIDADE HORIZONTAL VS. VERTICAL SOB RESTRIÇÃO DE RECURSOS

NOME	RA

Atividades Práticas Supervisionadas, apresentada à UNIP – Campus Araraquara/SP, referente ao 7º/8º Semestres, como parte dos requisitos para avaliação semestral, no Curso de Ciência da Computação.

ARARAQUARA/SP
2026

ÍNDICE

1 OBJETIVO E MOTIVAÇÃO DO TRABALHO 4
2 INTRODUÇÃO 6
2.1 Contexto do problema 6
2.2 O serviço de partida e seu colapso 6
2.3 A proposta de solução 7
2.4 Estrutura do documento 7
3 FUNDAMENTOS DAS TECNOLOGIAS DE ESCALABILIDADE E SISTEMAS DISTRIBUÍDOS 8
3.1 Escalabilidade vertical e horizontal 8
3.2 Isolamento de recursos em contêineres 8
3.3 Balanceamento de carga 9
3.4 Filas de mensagens e processamento assíncrono 9
3.5 Consistência, disponibilidade e tolerância à partição 10
3.6 Replicação mestre-escravo em bancos de dados relacionais 10
3.7 Persistência em lote 11
3.8 Testes de carga por taxa de chegada 11
3.9 Integridade do registro por âncora criptográfica 12
3.10 O índice Humidex 12
4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO 13
4.1 Escopo e restrições 13
4.2 Ferramentas e tecnologias 13
4.3 Etapas do desenvolvimento 14
4.4 Diagnóstico do baseline 14
4.5 Alternativas consideradas 15
4.6 Decisões de projeto e justificativas 15
4.7 Plano de testes e coleta de dados 16
4.8 Riscos e mitigações 17
4.9 Divisão de responsabilidades 17
5 PROJETO DA ARQUITETURA DISTRIBUÍDA 18
5.1 Visão geral 18
5.2 Módulos 18
5.3 Fluxos 19
5.4 Contrato de dados e integridade 20
5.5 Orçamento de CPU 20
5.6 Consistência e tolerância temporal 21
5.7 Limitações do projeto 21
6 RELATÓRIO COM AS LINHAS DE CÓDIGO 22
6.1 Serviço de borda e worker (código Go embutido no arquivo de composição) 22

6.2 Arquivo docker-compose.distribuido.yaml (sem o bloco Go) 25
7 APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO 30
7.1 Como executar 30
7.2 Resultados da matriz comparativa 30
7.3 Uso de recursos medido com docker stats 31
7.4 Comparação interna das duas versões do Cenário B 33
7.5 Discussão: resposta às indagações do enunciado 33
7.6 Aspectos de integridade do registro 35
7.7 Ameaças à validade 35
7.8 Conclusões 35
7.9 Trabalhos futuros 36
8 BIBLIOGRAFIA 37
9 FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS 39

1 OBJETIVO E MOTIVAÇÃO DO TRABALHO

Este trabalho tem como objetivo reprojetar a topologia de um serviço de ingestão e consulta de telemetria microclimática para que ele absorva rajadas de demanda sem perder nenhuma medição, mantendo o mesmo orçamento agregado de processamento do serviço original. O serviço de partida é um monolito síncrono escrito em Go, apoiado em um banco PostgreSQL, que funciona em regime nominal e entra em colapso quando muitos concentradores de campo descarregam seus buffers ao mesmo tempo. A restrição física do problema é fixa: nenhuma solução pode contar com mais de 0,80 de vCPU somadas, e cada instância da aplicação recebe exatamente 0,05 de vCPU.

O objetivo geral é, portanto, responder à seguinte pergunta: dado um volume fixo de ciclos computacionais, qual organização topológica extrai a maior eficiência útil de trabalho? Para respondê-la, o grupo compara dois cenários. O Cenário A é o monolito fornecido pelo docente, que serve de referência. O Cenário B é uma arquitetura distribuída de desenho livre, construída pela equipe e descrita integralmente em um arquivo complementar de composição de contêineres.

Os objetivos específicos são os seguintes:

- diagnosticar, a partir do código e da execução do baseline, quais características do monolito produzem o colapso sob rajada;
- projetar e implementar uma arquitetura distribuída que respeite o contrato HTTP, a cota individual de 0,05 de vCPU por instância da aplicação e o teto agregado de 0,80;
- garantir a integridade do registro, de modo que cada medição enviada seja persistida uma única vez e possa ser verificada por recálculo do hash;
- medir e comparar os dois cenários por meio de vazão, taxa de sucesso, taxa de persistência de registros e uso de recursos observado com o utilitário docker stats;
- documentar as decisões de projeto e as limitações da solução, de forma que outra equipe consiga reproduzir os experimentos.

A motivação do trabalho é dupla. Do ponto de vista técnico, o problema reúne em um único cenário controlado vários conceitos centrais de sistemas distribuídos: escalabilidade horizontal, balanceamento de carga, desacoplamento por filas, consistência eventual, replicação de bancos de dados e testes de carga por taxa de chegada. Do ponto de vista aplicativo, a infraestrutura de monitoramento ambiental tem função probatória e opera justamente nos momentos em que o dado é mais valioso, isto é, durante eventos atmosféricos extremos. Um serviço que perde dados nesses momentos falha no seu propósito, mesmo que responda rapidamente aos clientes.

Por fim, a restrição de recursos aproxima o exercício da prática profissional. Em ambientes reais, orçamentos de infraestrutura são finitos e a pergunta relevante raramente é “quanto hardware é necessário”, e sim “como usar melhor o hardware disponível”. O trabalho procura mostrar, com números medidos, que a forma de distribuir o trabalho pode ser mais determinante do que a quantidade de recursos.

2 INTRODUÇÃO

2.1 Contexto do problema

Redes de monitoramento microclimático são formadas por estações de coleta instaladas em áreas urbanas e periurbanas. Cada estação mede grandezas atmosféricas elementares, no caso a temperatura do ar, em graus Celsius, e a umidade relativa, em percentual. A combinação das duas serve de insumo para o índice de calor percebido Humidex, formulado por Masterson e Richardson (1979), que estima a pressão de vapor por meio de uma exponencial sobre a temperatura e corrige o valor pela umidade. Como o cálculo envolve uma exponencial e uma divisão em ponto flutuante, ele é intensivo em aritmética e, portanto, limitado por CPU.

Por restrição energética, esse cálculo é feito no servidor central e não no dispositivo de campo. Os sensores operam com bateria e captação solar, e o orçamento de energia disponível não comporta processamento contínuo. A cadeia completa tem, assim, quatro elos: a aquisição da grandeza no sensor, o transporte pela camada de comunicação, a ingestão e o processamento no servidor central e o consumo do dado agregado pelos sistemas de automação que tomam decisões.

A topologia da rede é hierárquica. Os sensores de borda enviam suas leituras a concentradores, chamados hubs, que acumulam os dados em um buffer e os descarregam em bloco quando o enlace com o servidor está disponível. Como a conectividade celular de baixa potência é intermitente, o servidor não observa um fluxo uniforme de leituras individuais, e sim lotes de tamanho variável, que no experimento vão de 10 a 60 medições. Sob determinadas condições atmosféricas, muitos hubs descarregam ao mesmo tempo, e o volume que chega em poucos segundos excede em ordens de grandeza o fluxo nominal.

2.2 O serviço de partida e seu colapso

O serviço de partida é um monolito síncrono. Cada requisição de ingestão precisa calcular o índice de todas as medições do lote, gravá-las em uma única transação e só então responder ao hub. Três características se combinam para produzir o colapso sob rajada. A primeira é o processamento síncrono, que mantém a conexão aberta até o fim da gravação. A segunda é o pool de conexões limitado, que obriga as requisições excedentes a esperar em fila dentro da aplicação. A terceira é a cota de CPU, de apenas 5% de uma vCPU, que torna o cálculo de um lote grande perceptivelmente lento.

O resultado é uma cascata. A fila interna cresce, as conexões ficam ocupadas por mais tempo, o kernel acumula conexões pendentes e o gerador de carga, que

injeta requisições a taxa fixa, passa a ter iterações descartadas. O sistema não apenas fica lento: ele deixa de absorver a demanda. A execução do baseline, realizada pelo grupo e descrita na Seção 9, confirmou esse comportamento: apenas 28,59% das medições enviadas foram persistidas.

2.3 A proposta de solução

A solução proposta separa as responsabilidades que o monolito mantinha juntas. A borda HTTP passa a ser atendida por várias instâncias leves, que validam o lote, calculam o hash e o Humidex e entregam o resultado a uma fila, respondendo imediatamente ao hub. Um consumidor independente retira os lotes da fila e os grava no banco em operações agrupadas. As consultas, que são muito mais numerosas do que as escritas, são atendidas por instâncias dedicadas que leem de uma réplica do banco. Um balanceador de carga na porta 8080 distribui as requisições para o conjunto correto de instâncias e preserva o contrato externo da aplicação.

Essa organização explora uma premissa do enunciado: a operação tolera que a medição esteja disponível para consulta com algum atraso, desde que não seja perdida. Essa tolerância permite trocar latência por vazão e adotar consistência eventual entre o instante da resposta ao hub e o instante da persistência efetiva (VOGELS, 2009).

2.4 Estrutura do documento

A Seção 3 apresenta os fundamentos teóricos de escalabilidade e sistemas distribuídos que sustentam as decisões do projeto. A Seção 4 descreve o plano de desenvolvimento, com ferramentas, etapas, alternativas consideradas e riscos. A Seção 5 detalha o projeto da arquitetura, seus módulos e fluxos. A Seção 6 reúne as linhas de código. A Seção 7 apresenta o sistema em funcionamento, a matriz comparativa de testes, as capturas de uso de recursos, a discussão dos resultados e as conclusões. Em seguida estão a bibliografia e a ficha de Atividades Práticas Supervisionadas.

3 FUNDAMENTOS DAS TECNOLOGIAS DE ESCALABILIDADE E SISTEMAS DISTRIBUÍDOS

3.1 Escalabilidade vertical e horizontal

Escalabilidade é a capacidade de um sistema de manter seu desempenho aceitável quando a demanda cresce. Há duas estratégias clássicas. A escalabilidade vertical aumenta a capacidade de uma única máquina, com mais CPU, memória ou disco. É simples de aplicar, pois não altera a estrutura do software, mas tem um teto físico e um custo que cresce mais rápido do que o ganho. A escalabilidade horizontal adiciona mais máquinas, ou instâncias, e distribui o trabalho entre elas. Ela depende de que o software possa ser dividido em partes que executem em paralelo, mas permite crescimento incremental e tolerância a falhas (TANENBAUM; VAN STEEN, 2007; KLEPPMANN, 2017).

No contexto deste trabalho, a escalabilidade vertical está bloqueada por regulamento: a cota de CPU de cada instância é fixa e o total é limitado a 0,80 de vCPU. Resta a escalabilidade horizontal, e o problema se transforma em decidir como distribuir um volume fixo de ciclos entre componentes especializados. A cota individual fixa isola a topologia como variável independente do experimento, pois qualquer diferença de resultado decorre da forma de organizar o trabalho e não de instâncias com capacidades distintas.

É útil lembrar o limite teórico do paralelismo. A lei de Amdahl (1967) estabelece que, se uma fração s do trabalho é inerentemente sequencial, o ganho máximo obtido com n unidades de processamento é 1 / (s + (1 − s) / n), que tende a 1 / s quando n cresce. Em um serviço de ingestão, a fração sequencial costuma estar no armazenamento compartilhado: transações que gravam as mesmas tabelas, confirmações de escrita no disco e contenção em índices. Por isso, simplesmente multiplicar instâncias da aplicação não basta; é preciso também reduzir o trabalho sequencial por medição, por exemplo agrupando gravações.

3.2 Isolamento de recursos em contêineres

Os experimentos executam em contêineres Docker. Um contêiner é um processo isolado pelo kernel Linux por meio de namespaces, que isolam a visão de recursos como processos e rede, e de cgroups, que limitam o consumo de CPU e memória. A cota de CPU declarada no arquivo de composição, como cpus: 0.05, é implementada pelo escalonador do kernel com controle de largura de banda: a cada período, o grupo de processos recebe um tempo máximo de execução e, ao esgotá-lo, é suspenso até o período seguinte. Isso explica um comportamento observado nos testes: uma instância limitada a 5% de vCPU pode ter picos instantâneos

medidos ligeiramente acima de 5%, pois o utilitário docker stats amostra em janelas que não coincidem com os períodos do escalonador.

O Docker Compose descreve a topologia de forma declarativa: serviços, redes, limites de recursos e dependências. Neste trabalho, o arquivo docker-compose.distribuido.yaml é o único local autorizado para extensões de engenharia, o que inclui proxies, filas, workers e bancos de apoio.

3.3 Balanceamento de carga

Um balanceador de carga recebe as requisições em um ponto único de entrada e as encaminha a um conjunto de servidores de aplicação. Seus objetivos são distribuir o trabalho, evitar que uma instância sature enquanto outras ficam ociosas e retirar de circulação instâncias que falham. Os algoritmos mais comuns são o rodízio simples, o rodízio ponderado e o de menor número de conexões ativas. O último é adequado quando as requisições têm custos desiguais, como acontece com lotes de 10 a 60 medições, porque envia o trabalho novo à instância menos ocupada (COULOURIS et al., 2013).

O HAProxy é um balanceador de camada 7 de alto desempenho, capaz de rotear por caminho da URL, de executar verificações periódicas de saúde e de manter uma fila de requisições quando todos os servidores atingem seu limite de conexões simultâneas (HAPROXY TECHNOLOGIES, [s.d.]). Essas propriedades são úteis aqui por três motivos: ele permite separar as rotas de escrita das rotas de leitura, mantém o contrato externo na porta 8080 e absorve variações de curto prazo com seu próprio enfileiramento.

3.4 Filas de mensagens e processamento assíncrono

Em uma arquitetura síncrona, o cliente espera o término de todo o processamento. Em uma arquitetura assíncrona, o servidor registra a intenção de processar o trabalho em uma fila e responde de imediato; um consumidor retira o trabalho da fila no seu próprio ritmo. Esse desacoplamento traz duas vantagens. A primeira é que a fila funciona como amortecedor: uma rajada que excede a capacidade de processamento por alguns segundos é absorvida e drenada depois, nos intervalos de calma. A segunda é que o produtor e o consumidor podem ser dimensionados de forma independente (KLEPPMANN, 2017).

A lei de Little (1961) ajuda a dimensionar esse comportamento. Ela afirma que o número médio de itens em um sistema é igual à taxa de chegada multiplicada pelo tempo médio de permanência, ou seja, $L = \lambda \times W$. Se a taxa de chegada supera temporariamente a taxa de serviço, a fila cresce; se em média a taxa de serviço supera a de chegada, a fila se esvazia. O sistema só é estável quando a capacidade

média de processamento excede a demanda média, e a fila serve para transformar picos em atrasos, em vez de perdas.

O Redis é um armazenamento de estruturas de dados em memória que oferece listas com inserção e remoção atômicas e bloqueantes, adequadas para filas simples e de baixo custo de CPU (REDIS LTD., [s.d.]). Em memória, a fila é rápida, mas volátil; por isso o Redis oferece o modo AOF (append-only file), em que cada comando de escrita é registrado em um arquivo de log, com sincronização configurável. O uso do AOF com sincronização a cada segundo reduz a janela de perda em caso de falha do contêiner a no máximo um segundo de comandos, ao custo de algum trabalho adicional de disco.

3.5 Consistência, disponibilidade e tolerância à partição

O teorema CAP, formalizado por Gilbert e Lynch (2002), afirma que um serviço distribuído não pode garantir simultaneamente consistência forte, disponibilidade e tolerância a partições de rede. Em termos práticos, quando há uma partição, o projeto precisa escolher entre recusar requisições para manter a consistência ou aceitá-las e reconciliar depois. Muitos sistemas adotam consistência eventual: na ausência de novas escritas, todas as réplicas convergem para o mesmo valor, mas leituras feitas durante a convergência podem observar valores antigos (VOGELS, 2009).

A atividade admite explicitamente essa escolha. A operação não exige que a medição esteja consultável imediatamente após a emissão; basta que a persistência se complete dentro de uma janela de tolerância. Essa premissa autoriza separar o instante da resposta ao hub do instante da gravação e, também, ler de uma réplica ligeiramente atrasada. O que não é negociável é a durabilidade: uma medição aceita e não persistida é uma perda permanente.

3.6 Replicação mestre-escravo em bancos de dados relacionais

Na replicação mestre-escravo, um servidor principal, o mestre, aceita as escritas e transmite suas alterações a um ou mais servidores secundários, os escravos ou réplicas, que as reproduzem e atendem apenas consultas. Em consequência, as leituras podem ser distribuídas e a carga de consulta deixa de competir com a de escrita. No PostgreSQL, a replicação por streaming envia continuamente o log de escrita antecipada (WAL) do mestre para a réplica, que o aplica em modo de espera ativa (hot standby), permitindo consultas somente de leitura enquanto replica (POSTGRESQL GLOBAL DEVELOPMENT GROUP, [s.d.]).

A replicação tem custos que precisam ser considerados. O mestre gasta CPU para enviar o WAL, a réplica gasta CPU para reaplicá-lo, e a réplica pode ficar

atrasada em relação ao mestre, o que é chamado de atraso de replicação. Em um orçamento fixo de CPU, a réplica só vale a pena se a economia de CPU no mestre, obtida ao tirar as leituras dele, compensar o gasto adicional. Este é exatamente o tipo de compromisso que a atividade pede que o grupo avalie.

3.7 Persistência em lote

Em um banco relacional, cada comando enviado tem um custo fixo, composto por ida e volta pela rede, análise do comando e, ao final de cada transação, a confirmação de escrita no disco. Quando se grava uma medição por vez, o custo fixo é pago uma vez por medição. Quando se agrupam muitas medições em um único comando, o custo fixo é diluído. No PostgreSQL, a função unnest permite transformar vetores recebidos como parâmetros em linhas de uma tabela e inseri-las em um único comando. A cláusula ON CONFLICT DO NOTHING torna a inserção idempotente em relação a uma restrição de unicidade: uma segunda tentativa de inserir o mesmo par de chaves é ignorada, sem erro.

Outro parâmetro relevante é a confirmação síncrona de escrita. Com synchronous_commit ligado, o banco só confirma uma transação depois que o WAL foi gravado no disco. Com synchronous_commit desligado, a confirmação ocorre antes da gravação em disco, o que reduz muito a latência da transação. A troca é que, em caso de falha do servidor, as transações confirmadas nos instantes anteriores podem ser perdidas, embora o banco permaneça consistente. Como essa é uma troca entre durabilidade e vazão, ela deve ser tratada como decisão consciente e documentada.

3.8 Testes de carga por taxa de chegada

Em um teste de laço fechado, um número fixo de clientes repete requisições o mais rápido possível, e cada cliente só envia a próxima requisição depois de receber a resposta anterior. Nesse modelo, um servidor lento recebe menos carga, e o problema fica escondido. Em um teste de laço aberto, a taxa de chegada é imposta de fora e independe do tempo de resposta; se o sistema não acompanha, o excedente é descartado ou espera. Schroeder, Wierman e Harchol-Balter (2006) demonstram que os dois modelos podem levar a conclusões muito diferentes sobre o mesmo sistema.

O gerador de carga usado na atividade é o k6, que implementa o modelo aberto com executores de taxa constante e de taxa em rampa (GRAFANA LABS, [s.d.]). As iterações que o gerador não consegue iniciar por falta de recursos livres no lado do cliente, ou porque o servidor segura as conexões, são contabilizadas

como descartadas. Essa métrica, junto com as requisições perdidas por limite de tempo, descreve a incapacidade de absorver a demanda.

3.9 Integridade do registro por âncora criptográfica

O registro de medições tem função probatória, e por isso cada medição recebe uma âncora criptográfica: o resumo SHA-256 da concatenação do identificador do sensor, do instante canônico da leitura e do pacote bruto recebido. O SHA-256 é uma função de resumo especificada no padrão FIPS 180-4 (NIST, 2015). Pequenas alterações na entrada produzem resumos completamente diferentes, o que torna a adulteração detectável por recálculo. A verificação independente se faz recalculando o resumo a partir dos campos devolvidos pela interface de consulta e comparando com o valor armazenado.

Como o instante entra no cálculo, sua representação textual não é livre. O contrato fixa o formato UTC com exatamente seis casas decimais, e qualquer variação, como três casas decimais ou um fuso local, geraria um resumo distinto para a mesma medição. Por isso, o sistema precisa transformar o timestamp recebido para a forma canônica antes de calcular o resumo e devolvê-lo nessa mesma forma na listagem de auditoria.

3.10 O índice Humidex

O Humidex é um índice canadense de desconforto térmico que combina temperatura e umidade (MASTERSON; RICHARDSON, 1979). Calcula-se a pressão de vapor de água e soma-se à temperatura do ar um termo proporcional ao excedente dessa pressão em relação a um valor de referência. Computacionalmente, o custo está na exponencial e na divisão em ponto flutuante. No projeto, o índice é calculado por medição na camada de borda, o que mantém a carga computacional real do problema e a distribui entre várias instâncias.

4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO

4.1 Escopo e restrições

O desenvolvimento seguiu o regulamento da atividade, que define quatro grupos de restrições. O primeiro é a imutabilidade do núcleo base: os arquivos do kit, entre eles o serviço de referência, o gerador de carga, o avaliador e os orquestradores de teste, não podem ser alterados. O segundo é a cota individual: toda instância da aplicação declara cpus: 0.05. O terceiro é o teto agregado de 0,80 de vCPU somando todos os serviços do arquivo distribuído. O quarto é o isolamento do código: qualquer extensão deve constar estritamente no arquivo docker-compose.distribuido.yaml.

O contrato externo também é fixo. O serviço de entrada deve ser publicado na porta 8080 e alcançável pela rede aps-net com o alias aps-app. Nenhum volume pode ser do tipo external, as rotas devem responder em JSON sem autenticação e o par formado pelo identificador do sensor e pelo instante da leitura deve ser único. As infrações desclassificatórias, como ultrapassar o teto de CPU, alterar a porta ou mockar o cálculo, resultam em nota zero e foram tratadas como requisitos de projeto.

4.2 Ferramentas e tecnologias

A Tabela 1 lista as ferramentas empregadas, com as versões efetivamente utilizadas nos experimentos.

Tabela 1 – Ferramentas e tecnologias empregadas

Ferramenta	Versão / imagem	Função no projeto
Docker Engine e Compose	Engine 29.7.2; Compose 5.4.0	Execução dos contêineres, redes e limites de CPU e memória
Go	golang:1.25-alpine	Linguagem do monolito e do serviço de borda (ingest e worker)
PostgreSQL	postgres:16-alpine	Banco de dados: mestre de escrita e réplica de leitura
Redis	redis:7-alpine	Fila de lotes entre a borda e o worker
HAProxy	haproxy:2.9-alpine	Balanceador e roteador de rotas na porta 8080
k6	grafana/k6:0.50.0	Gerador de carga por taxa de chegada
pgx e go-redis	pgx v5.7.2; go-redis v9.7.0	Bibliotecas Go de acesso ao PostgreSQL e ao Redis
docker stats	Docker CLI	Coleta de uso de CPU e memória durante os testes
PowerShell	Windows PowerShell 5.1	Script coletar_stats.ps1 e execução dos orquestradores

Fonte: elaborado pelos autores (2026).

4.3 Etapas do desenvolvimento

O trabalho foi organizado em oito etapas sequenciais, resumidas na Tabela 2. A ordem é importante: o diagnóstico do baseline precede o projeto, e as medições de cada versão precedem os ajustes seguintes, de modo que as decisões sejam apoiadas por dados.

Tabela 2 – Etapas do desenvolvimento e entregas

Etapa	Atividade	Entrega
1	Leitura do enunciado, do regulamento e do kit base	Lista de restrições e de infrações desclassificatórias
2	Análise do código do monolito, do gerador de carga e do avaliador	Entendimento do contrato, do manifesto e do cálculo da nota
3	Execução do Cenário A (baseline)	Resultados do baseline e capturas de uso de recursos
4	Diagnóstico dos gargalos do baseline	Hipóteses de projeto para o Cenário B
5	Projeto da arquitetura distribuída	Topologia, orçamento de CPU e contratos internos
6	Implementação do docker-compose.distribuido.yaml	Arquivo de composição validado com docker compose config
7	Execução do Cenário B e ajustes (sem réplica, depois com réplica)	Resultados, matriz comparativa e capturas de uso de recursos
8	Documentação, relatório técnico e apresentação	Este documento e a demonstração do sistema

Fonte: elaborado pelos autores (2026).

4.4 Diagnóstico do baseline

A análise do código do monolito identificou onde o tempo de CPU e de entrada e saída é gasto. Para cada medição de um lote, o serviço executa uma sequência de quatro comandos SQL dentro de uma única transação: a criação de um ponto de salvamento, a inserção da medição, a liberação do ponto de salvamento e a atualização do índice de calor do sensor. Um lote de 60 medições gera, portanto, cerca de 240 comandos em uma transação, cada um com ida e volta pela rede de contêineres. Como a transação só termina ao final do lote, a conexão do pool fica ocupada durante todo esse tempo.

Com o pool limitado, as requisições excedentes esperam na aplicação. A aplicação, com 5% de vCPU, gasta parte dos poucos ciclos disponíveis no tratamento de JSON, no cálculo do hash e do Humidex, no protocolo de comunicação com o banco e no próprio escalonamento de goroutines. A execução

do baseline mostrou o efeito: 69,85% dos POST e 61,92% dos GET excederam o limite de tempo do cliente, e 726 iterações foram descartadas pelo gerador de carga.

Os dados de uso de recursos do baseline mostram ainda que o PostgreSQL chegou a 50,7% de uso de uma vCPU. Esse valor é bem superior ao previsto no enunciado para o banco do baseline, o que decorre de uma divergência entre o arquivo de composição da pasta de trabalho, que dá 0,8 de vCPU ao banco, e o do enunciado, que indica 0,08. A divergência está registrada na Seção 7.7 como ameaça à validade da comparação.

4.5 Alternativas consideradas

Antes de implementar, o grupo avaliou diferentes estratégias, comparadas na Tabela 3. Apenas as duas últimas versões do Cenário B foram executadas; as demais foram descartadas por análise, e não por medição, o que se registra aqui para que o leitor saiba a diferença.

Tabela 3 – Alternativas de arquitetura consideradas

Alternativa	Argumento	Situação
Apenas balanceador e várias cópias do monolito	Multiplica as instâncias, mas todas disputam o mesmo banco e mantêm o processamento síncrono; a fração sequencial (lei de Amdahl) continua no banco	Descartada por análise
Pool de conexões externo (por exemplo, PgBouncer)	Reduz conexões no banco, mas não reduz o número de comandos por medição e consome CPU do orçamento	Descartada por análise
Fila (Redis), worker e gravação em lote	Responde antes de gravar, absorve a rajada e reduz drasticamente os comandos por medição	Adotada (v1 e v2)
Apache Kafka ou RabbitMQ no lugar do Redis	Mais garantias de entrega, mas maior consumo de memória e CPU, difícil de acomodar em 0,05 de vCPU por serviço	Descartada por análise
Réplica de leitura (mestre-escravo)	Retira as leituras do banco de escrita; custo de CPU da replicação e possível atraso	Adotada na v2, após orientação do professor
Cache de leitura em memória	Reduziria consultas ao banco, mas exigiria um serviço de leitura próprio e introduziria outra camada de inconsistência	Não implementada

Fonte: elaborado pelos autores (2026).

4.6 Decisões de projeto e justificativas

- Roteamento por caminho no balanceador: separar POST /telemetria das demais rotas permite dimensionar escrita e leitura de forma independente e usar o código original do monolito na leitura, sem alteração.

- Cálculo do hash e do Humidex na borda: preserva a carga computacional real exigida pelo regulamento e a distribui entre as instâncias de borda, evitando que o worker, que é único, se torne gargalo de CPU.
- Fila no Redis com AOF: oferece baixo custo de CPU e baixa latência, com durabilidade razoável por meio do registro dos comandos de escrita a cada segundo.
- Worker com gravação em lote: o worker retira até 100 mensagens por vez e as grava em dois comandos (um para as medições e outro para os índices), na mesma transação, com inserção idempotente.
- Repetição até gravar: o worker repete a gravação até que ela tenha sucesso, pois a mensagem já saiu da fila e não pode ser perdida.
- Réplica por streaming na v2: após a orientação do professor, a leitura passou a usar uma réplica, o que alivia o banco de escrita.
- Desligamento da confirmação síncrona de escrita no mestre: reduz o custo de confirmação por transação, ao preço de uma janela de perda em caso de falha do banco, registrada como limitação.
- Serviço de apoio db-init com cota própria de 0,01: executa uma vez e encerra, mas entra na soma para evitar qualquer dúvida na auditoria do orçamento.

4.7 Plano de testes e coleta de dados

Os testes seguem o procedimento oficial da atividade, executado pelo orquestrador run_benchmark.bat. A execução tem duas fases isoladas. Na primeira, o sistema sob teste sobe, aguarda a rota de saúde e recebe a carga do k6 por cerca de 145 segundos, com fluxo basal de 4 lotes por segundo na escrita e 50 consultas por segundo na leitura, e três picos que multiplicam essas taxas por 3. Na segunda fase, o avaliador observa a contagem de registros até estabilizar, baixa todas as medições pela rota paginada, recalcula os hashes e cruza o resultado com o manifesto de tudo o que foi despachado.

Para coletar o uso de recursos exigido, o grupo criou o script coletar_stats.ps1. Ele executa um cenário e, em paralelo, amostra o docker stats aproximadamente a cada segundo durante toda a execução, gravando um arquivo CSV. As métricas registradas são a vazão em requisições por segundo, a taxa de sucesso, as contagens de integridade (divergentes, fantasmas, perdidas e duplicatas), a taxa de persistência e o uso de CPU por serviço. As métricas de latência são apenas informativas, pois o avaliador oficial as desconsidera.

A nota é proporcional à taxa de persistência, isto é, ao número de registros gravados dividido pelo número enviado. O enunciado também cita o Índice de

Eficiência de Vazão, mas não fornece a fórmula. O grupo adotou a seguinte definição: IEV é a vazão útil, em requisições por segundo com sucesso, dividida pela soma das cotas de CPU alocadas, em vCPU. A definição precisa ser confirmada com o professor.

4.8 Riscos e mitigações

Tabela 4 – Riscos identificados e mitigações

Risco	Consequência	Mitigação
Soma de CPU acima de 0,80	Desclassificação com nota zero	Conferência da soma por script a cada alteração; total final de 0,79
Perda de lote por queda do worker após retirar da fila	Medições perdidas	Retentativa até gravar; janela residual documentada como limitação
Perda de dados na queda do Redis	Medições perdidas	AOF com sincronização a cada segundo
Atraso da réplica na auditoria	Contagem inferior à real	O avaliador espera a contagem estabilizar antes de verificar
Ingest com pouca folga de CPU	Degradação em cargas maiores	Pico medido de 4,7% sobre 5%; ajustar o número de instâncias se a carga crescer
Divergência entre o kit da pasta e o enunciado	Comparação injusta com o baseline	Divergência declarada e registrada como ameaça à validade
Falha de compilação do código embutido	Impossibilidade de subir o Cenário B	Compilação em estágio de build isolado do Docker; testada na primeira execução

Fonte: elaborado pelos autores (2026).

4.9 Divisão de responsabilidades

A Tabela 5 deve ser preenchida pelo grupo com os nomes dos integrantes, de acordo com a divisão real do trabalho.

Tabela 5 – Divisão de responsabilidades da equipe

Integrante (nome / RA)	Responsabilidade	Etapas

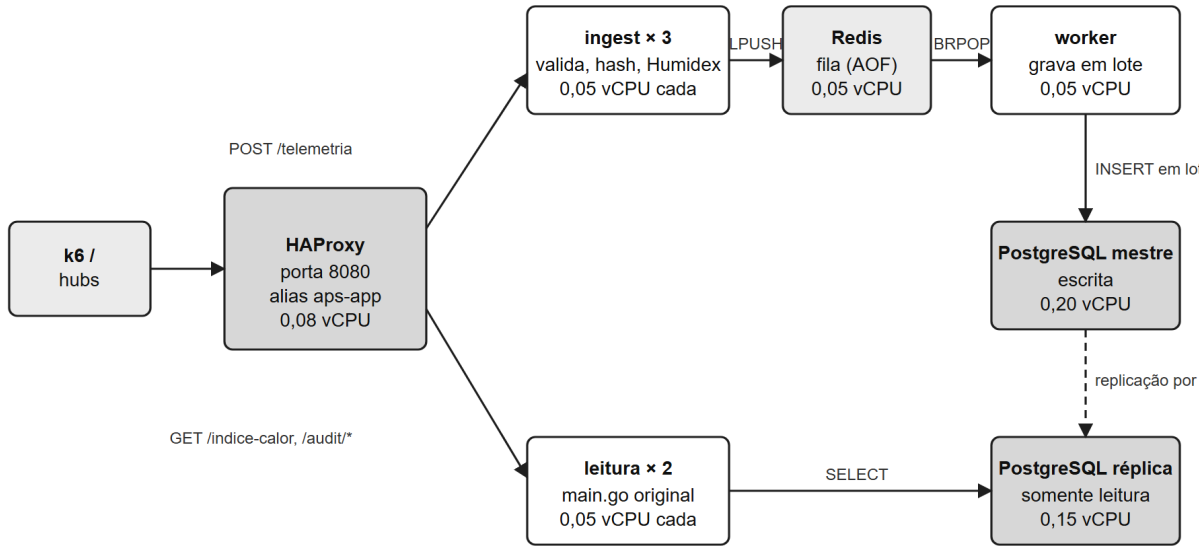
Fonte: elaborado pelos autores (2026).

5 PROJETO DA ARQUITETURA DISTRIBUÍDA

5.1 Visão geral

A arquitetura final, representada na Figura 1, tem oito tipos de componentes. Todas as requisições externas entram pelo HAProxy, que publica a porta 8080 no hospedeiro e declara o alias aps-app na rede aps-net. Os demais componentes ficam em uma rede interna do próprio arquivo de composição e não são acessíveis à plataforma de teste. As requisições de escrita seguem para a camada de borda, que alimenta a fila, que é consumida pelo worker, que grava no banco mestre. As requisições de leitura e de auditoria seguem para instâncias de leitura, que consultam a réplica.

Figura 1 – Arquitetura distribuída do Cenário B (v2)



Fonte: elaborado pelos autores (2026).

5.2 Módulos

HAProxy. Recebe todo o tráfego na porta 8080 com 0,08 de vCPU. As requisições cujo caminho começa por /telemetria são encaminhadas ao grupo de instâncias de borda; todas as demais, ao grupo de leitura. Ambos os grupos usam o algoritmo de menor número de conexões ativas, verificação periódica de saúde pela rota /health e descoberta das instâncias pelo DNS interno do Docker, por meio de uma definição de servidores em modelo (server-template). Assim, o número de instâncias pode mudar sem reescrever a configuração do balanceador, e cada instância tem um limite de conexões simultâneas para que o excedente aguarde na fila do próprio balanceador.

Instâncias de borda (ingest). São três instâncias de 0,05 de vCPU do serviço escrito em Go. Cada instância valida o corpo da requisição, rejeitando JSON malformado e lote vazio com código 400, e processa cada medição: converte o instante para a forma canônica com seis casas decimais, rejeita medições sem identificador, instante válido ou pacote bruto, calcula o SHA-256 e o Humidex e monta uma mensagem com as medições válidas do lote. A mensagem é inserida na fila do Redis, e a instância responde com código 202 e um objeto com os campos total_recebido, total_persistido, total_rejeitado e total_duplicado.

Fila (Redis). Uma instância de 0,05 de vCPU com persistência AOF a cada segundo e política de memória sem remoção de itens, de modo que, se a fila atingisse o limite de memória, a inserção falharia, e a borda responderia com erro em vez de descartar dados silenciosamente.

Worker. Uma instância de 0,05 de vCPU do mesmo serviço em Go, em modo consumidor. Ele bloqueia à espera de uma mensagem, retira em seguida até outras 99 se houver, une as medições e as grava no mestre em uma transação com dois comandos: uma inserção das medições por meio de unnest com ON CONFLICT DO NOTHING e uma inserção ou atualização do índice de calor de cada sensor, mantendo o último valor do conjunto. Se a gravação falha, ele a repete até ter sucesso.

Instâncias de leitura. São duas instâncias de 0,05 de vCPU executando o serviço de referência, sem nenhuma alteração do código, com conexão à réplica. Elas atendem /indice-calor, /audit/contagem, /audit/medidas e /health. Usar o código original na leitura garante que o formato de resposta continue idêntico ao do contrato.

Banco mestre e réplica. O mestre, com 0,20 de vCPU, recebe as escritas do worker, com WAL em nível de réplica e confirmação síncrona desligada. A réplica, com 0,15 de vCPU, é criada por uma cópia base do mestre (pg_basebackup) e acompanha o WAL em modo de espera ativa. O serviço db-init aplica o esquema do arquivo init.sql no mestre antes de a réplica ser clonada, e a cópia base já o inclui.

5.3 Fluxos

O fluxo de escrita ocorre nas seguintes etapas:

1. o gerador de carga ou o hub envia POST /telemetria ao endereço aps-app:8080;
2. o HAProxy escolhe a instância de borda com menos conexões ativas;
3. a instância valida o lote, canonicaliza o instante, calcula o hash e o Humidex de cada medição, e insere uma mensagem na fila;
4. a instância responde 202 ao cliente, sem esperar o banco;

5. o worker retira um conjunto de mensagens da fila e o grava no mestre em lote;
6. o mestre envia o WAL à réplica, que reproduz as alterações.

O fluxo de leitura é mais simples: o HAProxy encaminha GET /indice-calor/{sensor} ou GET /audit/* a uma instância de leitura, que consulta a réplica e devolve a resposta em JSON, com código 404 se o sensor ainda não tem medição.

5.4 Contrato de dados e integridade

A tabela medicoes mantém uma restrição de unicidade sobre o par sensor e instante, o que garante que duas linhas com os mesmos valores nunca coexistam. A inserção idempotente com ON CONFLICT DO NOTHING significa que, mesmo que uma mensagem seja processada duas vezes, por exemplo em uma retentativa, não haverá duplicata. O hash é calculado na borda sobre o identificador do sensor, o instante canônico e o pacote bruto, sem separadores, e é armazenado junto com a medição. A rota /audit/medidas devolve o instante no mesmo formato canônico, de modo que o avaliador consegue recalcular o hash e confirmar que o valor armazenado é o mesmo.

A rota de listagem usa paginação por cursor sobre o identificador sequencial, e não deslocamento. O cursor ancora a posição em um registro específico e permanece consistente sob inserção concorrente. A ausência de cursor na resposta indica o fim do conjunto.

5.5 Orçamento de CPU

A Tabela 6 mostra o orçamento de CPU do Cenário B na versão final. A soma é de 0,79 de vCPU, abaixo do teto de 0,80, e todas as instâncias da aplicação declaram 0,05.

Tabela 6 – Orçamento de CPU do Cenário B (v2)

Serviço	Instâncias × cota	CPU	Papel
haproxy	1 × 0,08	0,08	Balanceador
redis	1 × 0,05	0,05	Fila
postgres (mestre)	1 × 0,20	0,20	Escrita
postgres-replica	1 × 0,15	0,15	Leitura
worker	1 × 0,05	0,05	Consumidor da fila
ingest	3 × 0,05	0,15	Borda de escrita
leitura	2 × 0,05	0,10	Borda de leitura
db-init	1 × 0,01	0,01	Aplica o esquema
Total		0,79	Teto: 0,80

Fonte: elaborado pelos autores (2026).

5.6 Consistência e tolerância temporal

O sistema oferece consistência eventual em dois pontos. O primeiro é a separação entre a resposta ao hub e a gravação, mediada pela fila. O segundo é o atraso da réplica de leitura em relação ao mestre. O primeiro é limitado pela capacidade de vazão do worker e o segundo pela capacidade de reaplicação do WAL; nos testes, os dois ficaram abaixo do tempo de espera do avaliador, o que se confirma pela ausência de medições perdidas e pelo contador de leituras sem dado igual a zero. A janela de tolerância do enunciado, que é de dezenas de segundos, comporta com folga esse atraso.

5.7 Limitações do projeto

- Com a confirmação síncrona desligada, uma queda do mestre pode perder as últimas transações confirmadas, ainda que o banco permaneça consistente.
- O campo total_persistido da resposta de ingestão informa o número de medições aceitas e enfileiradas, e não o das já gravadas, porque a persistência é assíncrona.
- Se o worker falhar entre retirar um lote da fila e gravá-lo, o lote é perdido. A janela é pequena, mas não é nula; o uso de uma lista de processamento com confirmação eliminaria essa janela e fica como melhoria.
- A instância de borda tem folga pequena de CPU. Para cargas maiores, é necessário redistribuir o orçamento.
- A topologia usa um único worker e um único Redis, que são pontos únicos de falha. Em produção, seriam replicados.

6 RELATÓRIO COM AS LINHAS DE CÓDIGO

Esta seção reúne o código desenvolvido pelo grupo. Os arquivos do kit base, entre eles o main.go do monolito, o script.js do gerador de carga e o avaliar_justo.go, não foram modificados e por isso não são reproduzidos. Os dois listados a seguir fazem parte do arquivo docker-compose.distribuido.yaml. O código está apresentado em fonte menor e com espaçamento simples para caber no limite de páginas desta seção.

6.1 Serviço de borda e worker (código Go embutido no arquivo de composição)

```
package main

// edge: modo "ingest" (HTTP -> fila Redis) e modo "worker" (fila -> Postgres).

import (
  "context"
  "crypto/sha256"
  "encoding/hex"
  "encoding/json"
  "fmt"
  "log"
  "math"
  "net/http"
  "os"
  "strconv"
  "time"

  "github.com/jackc/pgx/v5/pgxpool"
  "github.com/redis/go-redis/v9"
)

const queueKey = "aps:ingest"

type input struct {
  SensorID  string `json:"sensor_id"`
  Timestamp string `json:"timestamp"`
  Metrics   struct {
    Microclimate struct {
      Temp float64 `json:"temperature_celsius"`
      Hum  float64 `json:"relative_humidity_percentage"`
    } `json:"microclimate"`
  } `json:"metrics"`
  Raw string `json:"raw_telemetry_packet"`
}

type batchInput struct {
  Measurements []input `json:"measurements"`
}

// rec e a mensagem da fila: ja carrega hash e Humidex calculados na borda.
type rec struct {
  S string  `json:"s"`
  T string  `json:"t"`
  R string  `json:"r"`
  H string  `json:"h"`
  X float64 `json:"x"`
}

func envInt(k string, d int) int {
  if v, err := strconv.Atoi(os.Getenv(k)); err == nil && v > 0 {
    return v
  }
  return d
}

func parseTS(s string) (time.Time, error) {
```

```
  for _, l := range []string{"2006-01-02T15:04:05.000000Z", time.RFC3339Nano,
time.RFC3339} {
    if t, err := time.Parse(l, s); err == nil {
      return t.UTC(), nil
    }
  }
  return time.Time{}, fmt.Errorf("timestamp invalido")
}

func canonical(t time.Time) string {
  return t.UTC().Format("2006-01-02T15:04:05.000000") + "Z"
}

func humidex(t, h float64) float64 {
  e := 6.112 * math.Exp((17.67*t)/(t+243.5)) * (h / 100.0)
  return math.Round((t+0.5555*(e-10.0))*100) / 100
}

func writeJSON(w http.ResponseWriter, code int, v interface{}) {
  w.Header().Set("Content-Type", "application/json; charset=utf-8")
  w.WriteHeader(code)
  json.NewEncoder(w).Encode(v)
}

func newRedis() *redis.Client {
  return redis.NewClient(&redis.Options{Addr: os.Getenv("REDIS_ADDR"), PoolSize: 20})
}

// ---------------- modo ingest ----------------

func runIngest() {
  rdb := newRedis()
  mux := http.NewServeMux()
  mux.HandleFunc("GET /health", func(w http.ResponseWriter, r *http.Request) {
    ctx, c := context.WithTimeout(r.Context(), 2*time.Second)
    defer c()
    if rdb.Ping(ctx).Err() != nil {
      writeJSON(w, 503, map[string]string{"status": "fila indisponivel"})
      return
    }
    writeJSON(w, 200, map[string]string{"status": "pronto"})
  })
  mux.HandleFunc("POST /telemetria", func(w http.ResponseWriter, r *http.Request) {
    var b batchInput
    if err := json.NewDecoder(r.Body).Decode(&b); err != nil {
      writeJSON(w, 400, map[string]string{"erro": "corpo JSON invalido"})
      return
    }
    if len(b.Measurements) == 0 {
      writeJSON(w, 400, map[string]string{"erro": "lote vazio"})
      return
    }
    recs := make([]rec, 0, len(b.Measurements))
    rejected := 0
    for _, m := range b.Measurements {
      ts, err := parseTS(m.Timestamp)
      if err != nil || m.SensorID == "" || m.Raw == "" {
        rejected++
        continue
      }
      c := canonical(ts)
      sum := sha256.Sum256([]byte(m.SensorID + c + m.Raw))
      recs = append(recs, rec{S: m.SensorID, T: c, R: m.Raw,
        H: hex.EncodeToString(sum[:]),
        X: humidex(m.Metrics.Microclimate.Temp, m.Metrics.Microclimate.Hum)})
    }
    if len(recs) > 0 {
      payload, _ := json.Marshal(recs)
      ctx, c := context.WithTimeout(r.Context(), 3*time.Second)
      defer c()
      if err := rdb.LPush(ctx, queueKey, payload).Err(); err != nil {
        writeJSON(w, 503, map[string]string{"erro": "fila indisponivel"})
        return
      }
    }
```

```
    // total_persistido = medicoes aceitas e enfileiradas (persistencia assincrona).
    writeJSON(w, 202, map[string]int{
      "total_recebido":   len(b.Measurements),
      "total_persistido": len(recs),
      "total_rejeitado":  rejected,
      "total_duplicado":  0,
    })
  })
  log.Println("ingest ouvindo em :8080")
  log.Fatal((&http.Server{Addr: ":8080", Handler: mux, ReadHeaderTimeout: 5 *
time.Second}).ListenAndServe())
}

// ---------------- modo worker ----------------

func flush(ctx context.Context, pool *pgxpool.Pool, all []rec) error {
  n := len(all)
  ss, ts, rs, hs := make([]string, n), make([]string, n), make([]string, n),
make([]string, n)
  last := map[string]float64{}
  for i, r := range all {
    ss[i], ts[i], rs[i], hs[i] = r.S, r.T, r.R, r.H
    last[r.S] = r.X
  }
  is, ix := make([]string, 0, len(last)), make([]float64, 0, len(last))
  for s, x := range last {
    is, ix = append(is, s), append(ix, x)
  }
  tx, err := pool.Begin(ctx)
  if err != nil {
    return err
  }
  defer tx.Rollback(ctx)
  if _, err := tx.Exec(ctx, `INSERT INTO medicoes (sensor_id, timestamp, raw_packet,
hash, criado_em)
    SELECT s, t::timestamptz, r, h, now()
    FROM unnest($1::text[], $2::text[], $3::text[], $4::text[]) AS u(s, t, r, h)
    ON CONFLICT (sensor_id, timestamp) DO NOTHING`, ss, ts, rs, hs); err != nil {
    return err
  }
  if _, err := tx.Exec(ctx, `INSERT INTO indices_calor (sensor_id, indice,
atualizado_em)
    SELECT s, x, now() FROM unnest($1::text[], $2::float8[]) AS u(s, x)
    ON CONFLICT (sensor_id) DO UPDATE SET indice = EXCLUDED.indice, atualizado_em =
now()`, is, ix); err != nil {
    return err
  }
  return tx.Commit(ctx)
}

func runWorker() {
  rdb := newRedis()
  cfg, err := pgxpool.ParseConfig(os.Getenv("DATABASE_URL"))
  if err != nil {
    log.Fatal(err)
  }
  cfg.MaxConns = 2
  ctx := context.Background()
  var pool *pgxpool.Pool
  for {
    if pool, err = pgxpool.NewWithConfig(ctx, cfg); err == nil && pool.Ping(ctx) == nil
{
      break
    }
    log.Println("aguardando banco:", err)
    time.Sleep(time.Second)
  }
  maxMsgs := envInt("WORKER_MAX_MSGS", 100)
  log.Println("worker consumindo", queueKey)
  for {
    first, err := rdb.BRPop(ctx, 2*time.Second, queueKey).Result()
    if err == redis.Nil {
      continue
    }
    if err != nil {
```

```
      log.Println("redis:", err)
      time.Sleep(time.Second)
      continue
    }
    msgs := []string{first[1]}
    if more, err := rdb.RPopCount(ctx, queueKey, maxMsgs-1).Result(); err == nil {
      msgs = append(msgs, more...)
    }
    var all []rec
    for _, m := range msgs {
      var rs []rec
      if json.Unmarshal([]byte(m), &rs) == nil {
        all = append(all, rs...)
      }
    }
    // Retenta ate gravar: a medicao ja saiu da fila, nao pode ser perdida.
    for {
      if err := flush(ctx, pool, all); err == nil {
        break
      } else {
        log.Println("falha ao gravar lote, retentando:", err)
        time.Sleep(time.Second)
      }
    }
  }
}

func main() {
  if os.Getenv("MODE") == "worker" {
    runWorker()
    return
  }
  runIngest()
}
```

6.2 Arquivo docker-compose.distribuido.yaml (sem o bloco Go)

```
# docker-compose.distribuido.yaml — CENARIO B (arquitetura distribuida)
#
# Topologia (tudo atras da porta 8080, contrato HTTP inalterado):
#
#   k6 --> haproxy:8080 (alias aps-app)
#            |-- POST /telemetria ------> ingest x5 --LPUSH--> redis --BRPOP--> worker -
-> postgres
#            |-- GET  /indice-calor, /audit/*, /health ---> leitura x2 (main.go
original) --> postgres-replica
#
#   postgres (MASTER, escrita) --replicacao em streaming--> postgres-replica (SLAVE,
somente leitura)
#
# Ideia: o cliente HTTP e isolado na borda. A rota de escrita valida, calcula
# hash e Humidex (CPU real, nada e simulado), enfileira no Redis e responde 202.
# Um worker consome a fila e grava em LOTE (1 INSERT com unnest para varias
# requisicoes), eliminando o gargalo do pool/commit por requisicao. As leituras
# ficam em instancias separadas, usando o main.go original sem alteracao.
#
# ORCAMENTO DE CPU (limits.cpus, instancias contadas individualmente):
#   haproxy          1 x 0.08 = 0.08
#   redis            1 x 0.05 = 0.05
#   postgres master  1 x 0.20 = 0.20
#   postgres replica 1 x 0.15 = 0.15
#   worker           1 x 0.05 = 0.05
#   ingest           3 x 0.05 = 0.15
#   leitura          2 x 0.05 = 0.10
#   db-init          1 x 0.01 = 0.01  (roda uma vez e encerra)
#   TOTAL                       0.79  (teto: 0.80)
#
# Regras: toda instancia da aplicacao (ingest, worker, leitura) usa
# exatamente cpus: '0.05'; porta publicada 8080; alias aps-app em aps-net;
# nenhum volume external. O codigo Go extra vive inline (dockerfile_inline).

x-edge-build: &edge-build
  context: .
  dockerfile_inline: |
```

```
    # syntax=docker/dockerfile:1
    FROM golang:1.25-alpine AS builder
    WORKDIR /src
    COPY <<'EOF' /src/main.go
    # ... (codigo Go do servico "edge": ver Secao 8.1) ...
    EOF
    RUN go mod init aps/edge \
     && go get github.com/jackc/pgx/v5@v5.7.2 github.com/redis/go-redis/v9@v9.7.0 \
     && go mod tidy \
     && CGO_ENABLED=0 GOOS=linux go build -trimpath -ldflags="-s -w" -o /out/edge
main.go
    FROM alpine:3.20
    RUN apk add --no-cache ca-certificates
    COPY --from=builder /out/edge /usr/local/bin/edge
    EXPOSE 8080
    ENTRYPOINT ["/usr/local/bin/edge"]

configs:
  haproxy_cfg:
    content: |
      global
        maxconn 8192
      defaults
        mode http
        timeout connect 3s
        timeout client 30s
        timeout server 30s
        timeout queue 20s
        option redispatch
        retries 2
      resolvers docker
        nameserver dns 127.0.0.11:53
        hold valid 2s
        accepted_payload_size 8192
      frontend fe
        bind *:8080
        acl is_ingest path_beg /telemetria
        use_backend ingest if is_ingest
        default_backend leitura
      backend ingest
        balance leastconn
        option httpchk GET /health
        default-server check inter 5s fall 3 rise 1 resolvers docker init-addr none
maxconn 100
        server-template i 3 ingest:8080
      backend leitura
        balance leastconn
        option httpchk GET /health
        default-server check inter 10s fall 3 rise 1 resolvers docker init-addr none
maxconn 50
        server-template l 2 leitura:8080

  pg_repl_init:
    content: |
      #!/bin/sh
      # libera conexoes de replicacao (o pg_hba padrao da imagem nao cobre
"replication")
      echo "host replication all all scram-sha-256" >> "$$PGDATA/pg_hba.conf"

services:
  postgres:
    image: postgres:16-alpine
    command:
      - postgres
      - -c
      - synchronous_commit=off
      - -c
      - max_connections=50
      - -c
      - shared_buffers=64MB
      - -c
      - max_wal_size=1GB
      - -c
      - wal_level=replica
      - -c
```

```
      - max_wal_senders=5
    environment:
      - POSTGRES_USER=iot
      - POSTGRES_PASSWORD=iot
      - POSTGRES_DB=telemetria
    configs:
      - source: pg_repl_init
        target: /docker-entrypoint-initdb.d/10-replicacao.sh
        mode: 0755
    deploy:
      resources:
        limits:
          cpus: '0.20'
          memory: 256M
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U iot -d telemetria"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 15s

  postgres-replica:
    image: postgres:16-alpine
    entrypoint: ["/bin/sh", "-c"]
    command:
      - |
        export PGPASSWORD=iot
        D=/var/lib/postgresql/data
        if [ ! -s $$D/PG_VERSION ]; then
          until su-exec postgres pg_basebackup -h postgres -U iot -D $$D -R -X stream;
do
            echo "aguardando master para pg_basebackup..."; sleep 2
          done
          chmod 700 $$D
        fi
        exec su-exec postgres postgres -c hot_standby=on -c max_connections=50 -c
shared_buffers=64MB
    deploy:
      resources:
        limits:
          cpus: '0.15'
          memory: 256M
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U iot -d telemetria"]
      interval: 5s
      timeout: 5s
      retries: 30
      start_period: 15s
    depends_on:
      db-init:
        condition: service_completed_successfully

  db-init:
    image: postgres:16-alpine
    environment:
      - PGPASSWORD=iot
    volumes:
      - ./init.sql:/init.sql:ro
    entrypoint: ["/bin/sh", "-c"]
    command:
      - |
        for i in $$(seq 1 30); do
          psql -h postgres -U iot -d telemetria -f /init.sql && exit 0
          sleep 1
        done
        echo "Falha ao aplicar init.sql" && exit 1
    deploy:
      resources:
        limits:
          cpus: '0.01'
          memory: 64M
    depends_on:
      postgres:
        condition: service_healthy
```

```
  redis:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes", "--appendfsync", "everysec", "--
maxmemory", "100mb", "--maxmemory-policy", "noeviction"]
    deploy:
      resources:
        limits:
          cpus: '0.05'
          memory: 128M
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 10

  ingest:
    build: *edge-build
    image: aps-edge:distribuido
    environment:
      - MODE=ingest
      - REDIS_ADDR=redis:6379
      - GOMAXPROCS=2
    deploy:
      replicas: 3
      resources:
        limits:
          cpus: '0.05'
          memory: 64M
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/health || exit 1"]
      interval: 15s
      timeout: 3s
      retries: 5
      start_period: 5s
    depends_on:
      redis:
        condition: service_healthy

  worker:
    build: *edge-build
    image: aps-edge:distribuido
    environment:
      - MODE=worker
      - REDIS_ADDR=redis:6379
      - DATABASE_URL=postgres://iot:iot@postgres:5432/telemetria?sslmode=disable
      - GOMAXPROCS=2
    deploy:
      replicas: 1
      resources:
        limits:
          cpus: '0.05'
          memory: 64M
    depends_on:
      redis:
        condition: service_healthy
      db-init:
        condition: service_completed_successfully

  leitura:
    build:
      context: .
      dockerfile: Dockerfile.monolito
    environment:
      - DATABASE_URL=postgres://iot:iot@postgres-replica:5432/telemetria?sslmode=disable
      - GOMAXPROCS=2
      - DB_MAX_CONNS=4
    deploy:
      replicas: 2
      resources:
        limits:
          cpus: '0.05'
          memory: 64M
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/health || exit 1"]
      interval: 30s
```

```
      timeout: 5s
      retries: 3
      start_period: 20s
    depends_on:
      postgres-replica:
        condition: service_healthy

  haproxy:
    image: haproxy:2.9-alpine
    configs:
      - source: haproxy_cfg
        target: /usr/local/etc/haproxy/haproxy.cfg
    ports:
      - "8080:8080"
    deploy:
      resources:
        limits:
          cpus: '0.08'
          memory: 64M
    networks:
      default:
      aps-net:
        aliases: [aps-app]
    depends_on:
      ingest:
        condition: service_healthy
      leitura:
        condition: service_healthy

networks:
  aps-net:
    external: true
```

7 APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO

7.1 Como executar

O sistema foi executado em um computador com Windows 11 e Docker Desktop. O orquestrador oficial executa os dois cenários em sequência e salva os artefatos em resultados/CenarioA e resultados/CenarioB. Para coletar também as amostras de docker stats, usa-se o script coletar_stats.ps1 com a letra do cenário:

```
docker network create aps-net
.\coletar_stats.ps1 A     # Cenário A: monolito
.\coletar_stats.ps1 B     # Cenário B: distribuído

# A cada rodada:
#   resultados\CenarioX\<data_hora>\report.json            (métricas do k6)
#   resultados\CenarioX\<data_hora>\resultado_registros.txt (auditoria)
#   resultados\docker_stats_CenarioX.csv                    (docker stats)
```

A verificação de conformidade do arquivo de composição foi feita com docker compose config, que aceitou o arquivo, e por conferência da soma das cotas de CPU. Antes da execução do Cenário B, as rotas descritas no Anexo A foram também exercitadas pelo próprio avaliador, que baixa o conjunto completo de medições pela rota paginada e recalcula todos os hashes.

Funcionalidades pedidas pelo enunciado, atendidas: rota de saúde com código 200 e 503; ingestão em lote com resposta por contagens; consulta do índice médio com código 404 quando não há medição; auditoria por contagem; listagem paginada por cursor; timestamp canônico com seis casas decimais; hash SHA-256 por medição; unicidade do par sensor e instante; porta 8080 e alias aps-app; ausência de volumes externos. Funcionalidades extras do grupo: replicação por streaming, persistência AOF da fila, descoberta dinâmica de instâncias no balanceador, verificações de saúde, script de coleta de uso de recursos e matriz comparativa automatizada a partir dos artefatos de teste.

7.2 Resultados da matriz comparativa

A Tabela 7 apresenta a matriz oficial, com os dois cenários. A carga foi a padrão do kit: fluxo basal de 4 lotes por segundo na escrita e 50 consultas por segundo na leitura, com pico de fator 3, e lotes de 10 a 60 medições, por cerca de 145 segundos.

Tabela 7 – Matriz comparativa oficial (Cenário A × Cenário B)

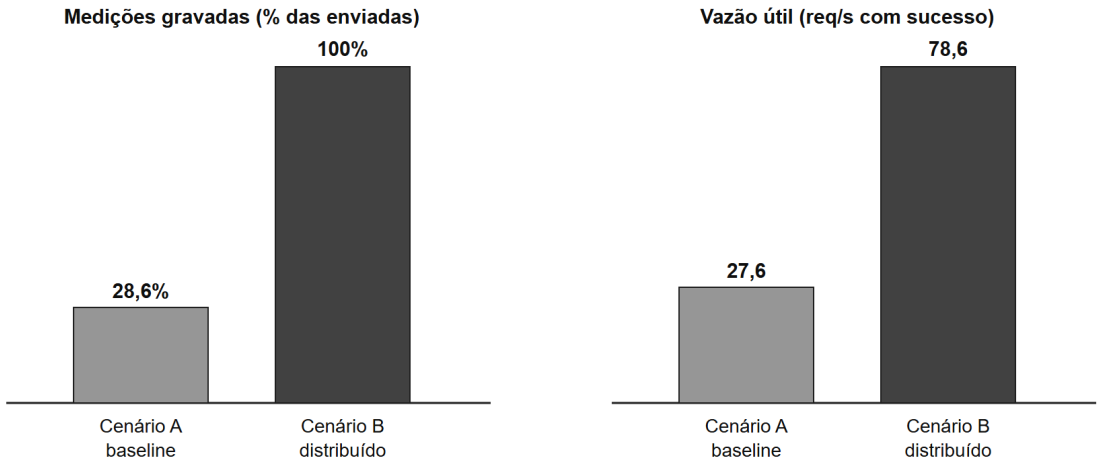
Métrica	Cenário A (baseline)	Cenário B (distribuído)
Vazão total (req/s completadas)	73,57	78,59
Vazão útil (req/s com sucesso)	27,57	78,59
Taxa de sucesso das requisições	37,47%	100,00%
POST /telemetria aceitos	249 de 826	844 de 844

Métrica	Cenário A (baseline)	Cenário B (distribuído)
GET /indice-calor válidos (2xx ou 404)	3.748 de 9.842	10.551 de 10.551
Iterações descartadas pelo k6	726	0
Medições enviadas	28.797	29.149
Medições gravadas	8.233	29.149
Medições perdidas	20.564	0
Taxa de persistência (gravados ÷ enviados)	28,59%	100,00%
Divergentes / fantasmas / duplicatas	0 / 0 / 0	0 / 0 / 0
Nota (10 × taxa de persistência)	2,86	10,00
Registros gravados por segundo	56,8	201,0
Latência p95 (ms)	5.001 (limite de tempo)	75,9
CPU alocada (soma de limits.cpus)	0,85 (nota 1)	0,79
Índice de Eficiência de Vazão (nota 2)	32,4	99,5

Fonte: elaborado pelos autores (2026).

Nota 1: o arquivo de composição do baseline entregue na pasta de trabalho dá 0,8 de vCPU ao Postgres e 0,05 à aplicação. O enunciado indica 0,08 para o banco. Usou-se o valor do arquivo. Nota 2: IEV é a vazão útil dividida pela soma das cotas de CPU, conforme a definição adotada pelo grupo na Seção 4.7.

Figura 2 – Persistência de registros e vazão útil por cenário



Fonte: elaborado pelos autores (2026).

7.3 Uso de recursos medido com docker stats

A Tabela 8 resume o pico de uso de CPU de cada serviço durante a execução, em percentual de uma vCPU, obtido do arquivo de amostras do docker

stats. Valores ligeiramente acima da cota se devem à janela de amostragem, como explicado na Seção 3.2.

Tabela 8 – Pico de CPU por serviço (% de 1 vCPU)

Serviço	Limite	Cenário A	B v1 (sem réplica)	B v2 (com réplica)
Aplicação (monolito)	5%	5,6%	—	—
PostgreSQL (mestre)	A: 80%; v1: 25%; v2: 20%	50,7%	25,0%	19,5%
PostgreSQL (réplica)	15%	—	—	12,3%
HAProxy	8%	—	7,7%	7,2%
Ingest (por instância)	5%	—	4,3%	4,7%
Leitura (por instância)	5%	—	5,7%	5,2%
Redis	5%	—	4,9%	4,2%
Worker	5%	—	3,4%	3,2%

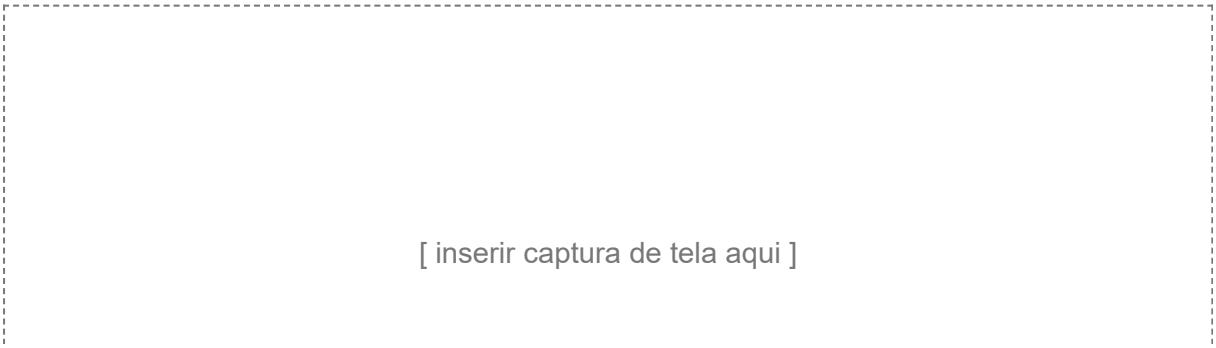
Fonte: elaborado pelos autores (2026).

Figura 3 – Captura do docker stats durante o patamar de pico do Cenário A (inserir captura de tela)



Fonte: elaborado pelos autores (2026).

Figura 4 – Captura do docker stats durante o patamar de pico do Cenário B (inserir captura de tela)



Fonte: elaborado pelos autores (2026).

Os dados brutos de todas as amostras estão nos arquivos CSV gerados pelo script de coleta, que acompanham o trabalho.

7.4 Comparação interna das duas versões do Cenário B

Além do Cenário A, o grupo executou duas versões do Cenário B, para avaliar o efeito da réplica de leitura. A versão 1 usa um único PostgreSQL de 0,25 de vCPU, cinco instâncias de borda e duas de leitura. A versão 2, adotada após a orientação do professor, divide o banco em mestre de 0,20 e réplica de 0,15, e reduz a borda a três instâncias para manter o orçamento de 0,79.

Tabela 9 – Comparação das versões do Cenário B

Métrica	B v1 (sem réplica)	B v2 (com réplica)
Instâncias de borda / leitura	5 / 2	3 / 2
Banco de dados	1 × 0,25	mestre 0,20 + réplica 0,15
Medições gravadas / enviadas	29.610 / 29.610	29.149 / 29.149
Taxa de persistência	100,00%	100,00%
Vazão útil (req/s)	78,57	78,59
Latência p95 (ms)	89,6	75,9
IEV	99,5	99,5
Pico de CPU do mestre	25,0% de 25% (saturado)	19,5% de 20%

Fonte: elaborado pelos autores (2026).

As duas versões atingiram 100% de persistência. O ganho observável da réplica está na folga do banco de escrita, que saiu do limite de CPU na v1 e ficou com margem na v2, e em uma latência p95 ligeiramente menor. Como cada versão foi medida em uma única rodada com stats, o grupo não afirma que uma supere estatisticamente a outra em persistência.

7.5 Discussão: resposta às indagações do enunciado

Colapso do baseline. A hipótese do Cenário A era o colapso da fila síncrona por saturação do pool de conexões e do kernel, com descarte de iterações sob a rajada. Os dados a confirmam: 726 iterações foram descartadas, 69,85% dos POST e 61,92% dos GET estouraram o limite de tempo de 5 segundos do cliente, e a

latência p95 foi de 5.001 ms, o que corresponde ao próprio limite de tempo. Apenas 28,59% das medições enviadas foram persistidas, e as 20.564 medições perdidas foram, na terminologia do avaliador, despachadas e não gravadas. O contador de integridade mostrou 0 divergentes, 0 fantasmas e 0 duplicatas, o que indica que o baseline não corrompeu os dados que gravou; ele apenas perdeu os que não conseguiu atender.

Isolamento da borda e absorção assíncrona. A hipótese do Cenário B era o isolamento do cliente HTTP na borda com absorção assíncrona da rajada. Ela se confirmou: nenhuma iteração foi descartada, 100% das requisições foram aceitas e 100% das medições foram gravadas, com a latência p95 de 75,9 ms. A resposta imediata ao hub, antes da gravação, é o que impede a formação de fila de conexões abertas. A fila no Redis e o worker em lote transformam o pico em atraso, que fica dentro da janela de tolerância.

Leitura sem dado. O contador leitura_sem_dado, que registra as consultas que retornaram 404, foi de 2.002 no Cenário A e de 0 no Cenário B. No A, o valor alto se deve à lentidão: muitas gravações não terminaram antes da consulta, e as que foram perdidas nunca existirão. No B, apesar de o índice ser atualizado de forma assíncrona e lido de uma réplica, o valor zero indica que o atraso entre a aceitação da medição e a atualização do índice foi menor do que o intervalo de uma consulta ao mesmo sensor nesse perfil de carga. Isso não significa que o atraso não exista, e sim que, neste experimento, ele não foi observável pela leitura. Em uma carga maior, o contador poderia crescer, o que o torna um bom indicador de atraso da fila.

Eficiência de uso do orçamento. Com 0,79 de vCPU alocadas, o Cenário B entregou 78,6 requisições úteis por segundo, contra 27,6 do baseline, e gravou cerca de 201 registros por segundo, contra 56,8. Em termos do IEV adotado pelo grupo, isso representa 99,5 contra 32,4, ou seja, cerca de três vezes mais trabalho útil por vCPU alocada. A organização topológica, e não a quantidade de CPU, explica a diferença: o Cenário A tinha, no arquivo da pasta, ainda mais CPU alocada do que o B.

Papel de cada técnica. Como o experimento não isolou cada técnica separadamente, a atribuição do ganho a cada componente é uma interpretação. Pela análise do baseline, a maior parte do ganho decorre de dois fatores: responder antes de gravar, que elimina a retenção de conexões, e gravar em lote, que reduz de cerca de 240 comandos por lote para dois por conjunto de lotes. O balanceamento entre várias instâncias de borda contribui para distribuir o cálculo do hash e do Humidex, e a réplica alivia o banco de escrita, como mostra a comparação entre v1 e v2.

7.6 Aspectos de integridade do registro

Em ambos os cenários, o avaliador não encontrou registros divergentes, fantasmas ou duplicados. Isso mostra que o contrato de integridade foi respeitado: o timestamp devolvido pela rota de auditoria está no formato canônico, e o hash recalculado coincide com o armazenado. No Cenário B, a unicidade do par sensor e instante é garantida pela restrição do banco e pela inserção idempotente, e a ausência de perdas é confirmada pelo cruzamento de cada hash do manifesto com a listagem completa recuperada pela API.

7.7 Ameaças à validade

- Número de rodadas: cada configuração foi medida em poucas rodadas, e a comparação entre v1 e v2 não permite conclusão estatística. Recomenda-se repetir cada cenário e usar a média.
- Divergência do kit: o arquivo de composição do baseline da pasta dá 0,8 de vCPU ao banco, e o enunciado indica 0,08. O resultado do Cenário A usou o arquivo da pasta, que dá mais recursos ao baseline do que o enunciado prevê; o efeito de usar 0,08 não foi medido.
- Definição do IEV: a fórmula não consta no enunciado, e a definição usada é uma proposta do grupo, a ser confirmada.
- Ambiente: a execução foi feita em um único computador pessoal, com o Docker Desktop. A ausência de variação entre máquinas reduz-se pelo uso de cotas de CPU, mas não desaparece.
- Carga única: a conclusão vale para o perfil de carga do kit; cargas maiores podem expor o limite de CPU das instâncias de borda.

7.8 Conclusões

O trabalho mostrou que, sob um orçamento fixo de CPU, a organização da topologia determina o resultado. O monolito síncrono, que atende bem em regime nominal, perdeu 71,4% das medições na rajada, enquanto a arquitetura distribuída, com o mesmo contrato HTTP e um orçamento menor de CPU (0,79 contra 0,85 do arquivo do baseline), gravou 100% das medições, sem divergência, fantasma nem duplicata. A fração de ganho vem de separar a borda da persistência, de gravar em lote e de usar uma réplica para tirar as leituras do banco de escrita.

O desenvolvimento deixou também lições metodológicas. A primeira é que medir antes de projetar evitou otimizações no lugar errado: o diagnóstico do baseline apontou o excesso de comandos por medição e a retenção de conexões. A segunda é que toda otimização tem um custo explícito que precisa ser registrado, como a confirmação assíncrona de escrita e a janela de perda em queda do worker. A

terceira é que o uso de recursos precisa ser observado e não presumido: o docker stats mostrou que o banco estava saturado na v1 e que a réplica aliviou essa pressão na v2, e mostrou também que a borda tem pouca folga.

7.9 Trabalhos futuros

- repetir as rodadas e calcular média e desvio-padrão por cenário;
- testar cargas maiores, para encontrar o ponto de saturação das instâncias de borda e do worker;
- trocar a retirada simples da fila por uma lista de processamento com confirmação, eliminando a janela de perda em queda do worker;
- avaliar um cache de leitura com validade curta, aproveitando a tolerância temporal da operação;
- avaliar mais de um worker, com particionamento da fila por sensor;
- executar a variante do baseline com 0,08 de vCPU no banco, para comparar com a do arquivo da pasta.

8 BIBLIOGRAFIA

AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. In: AFIPS SPRING JOINT COMPUTER CONFERENCE, 1967, Atlantic City. **Proceedings** [...]. New York: ACM, 1967. p. 483-485.

COULOURIS, G. et al. **Sistemas distribuídos**: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

GILBERT, S.; LYNCH, N. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. **ACM SIGACT News**, New York, v. 33, n. 2, p. 51-59, 2002.

GRAFANA LABS. **k6 documentation**. [S. l.]: Grafana Labs, [s.d.]. Disponível em: https://grafana.com/docs/k6/. Acesso em: ___ /___ /2026.

HAPROXY TECHNOLOGIES. **HAProxy documentation**. [S. l.]: HAProxy Technologies, [s.d.]. Disponível em: https://docs.haproxy.org/. Acesso em: ___ /___ /2026.

KLEPPMANN, M. **Designing data-intensive applications**: the big ideas behind reliable, scalable, and maintainable systems. Sebastopol: O'Reilly Media, 2017.

LITTLE, J. D. C. A proof for the queuing formula: L = λW. **Operations Research**, Catonsville, v. 9, n. 3, p. 383-387, 1961.

MASTERSON, J. M.; RICHARDSON, F. A. **Humidex**: a method of quantifying human discomfort due to excessive heat and humidity. Downsview: Environment Canada, Atmospheric Environment Service, 1979. (CLI 1-79).

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **FIPS PUB 180-4**: secure hash standard (SHS). Gaithersburg: NIST, 2015.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 documentation**: high availability, load balancing, and replication. [S. l.]: PostgreSQL Global Development Group, [s.d.]. Disponível em: https://www.postgresql.org/docs/16/high-availability.html. Acesso em: ___ /___ /2026.

REDIS LTD. **Redis documentation**. [S. l.]: Redis Ltd., [s.d.]. Disponível em: https://redis.io/docs/. Acesso em: ___ /___ /2026.

SCHROEDER, B.; WIERMAN, A.; HARCHOL-BALTER, M. Open versus closed: a cautionary tale. In: SYMPOSIUM ON NETWORKED SYSTEMS DESIGN AND IMPLEMENTATION, 3., 2006, San Jose. **Proceedings** [...]. Berkeley: USENIX Association, 2006. p. 239-252.

TANENBAUM, A. S.; VAN STEEN, M. **Sistemas distribuídos**: princípios e paradigmas. 2. ed. São Paulo: Pearson Prentice Hall, 2007.

UNIVERSIDADE PAULISTA. **Atividades práticas supervisionadas**: orientações para a disciplina, 7º e 8º semestres de Ciência da Computação, 2026/2. Araraquara: UNIP, 2026.

VOGELS, W. Eventually consistent. **Communications of the ACM**, New York, v. 52, n. 1, p. 40-44, 2009.

9 FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS

FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

NOME: _____ TURMA: _____ RA: _____

CURSO: Ciência da Computação CAMPUS: Araraquara SEMESTRE: _____ TURNO: _____

CÓDIGO DA ATIVIDADE: _____ SEMESTRE: _____ ANO GRADE: _____

DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
	Leitura e análise do enunciado, do regulamento e do kit base			
	Análise do código do monolito, do gerador de carga e do avaliador			
	Execução e medição do Cenário A (baseline)			
	Diagnóstico dos gargalos do baseline			
	Projeto da arquitetura distribuída (Cenário B)			
	Implementação do docker-compose.distribuido.yaml			
	Execução e ajuste do Cenário B (sem e com réplica) e coleta de docker stats			
	Elaboração da matriz comparativa de testes			
	Redação do relatório técnico e preparação da apresentação			

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS: _____ AVALIAÇÃO (Aprovado ou Reprovado): _____ NOTA: _____ DATA: ____/____/_____

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO:

_____